



Data Structure And Algorithms

Arrays

Introduction & Definition of an Array

- An array is a group of similar elements or data items of the same type collected at contiguous memory locations. In simple words, we can say that in computer programming, arrays are generally used to organize the same type of data. Data items stored in an array are known as elements.
- **Array Representation:**
- An array can be declared in various ways and in different languages. The important points that should be considered are as follows:
- The location or placing of each element has an index value.
- The index starts with 0.
- We can easily find any elements within this Array using the Index value.
- The length of the Array defines the capacity to store the elements. It is written like `x[100]`, which means the length of array `x` is specified by 100.
- Arrays are used to store multiple values in one single variable.
- It is easier to calculate the position of each element by simply adding an offset to a base value, i.e., the memory location of the first element of the array.

Array: Indexes 0 1 2 3 4
Values 1 3 8 23 99

Array: Indexes 0 1 2 3 4
Values a b c d e

Array can store only the same type of data items.

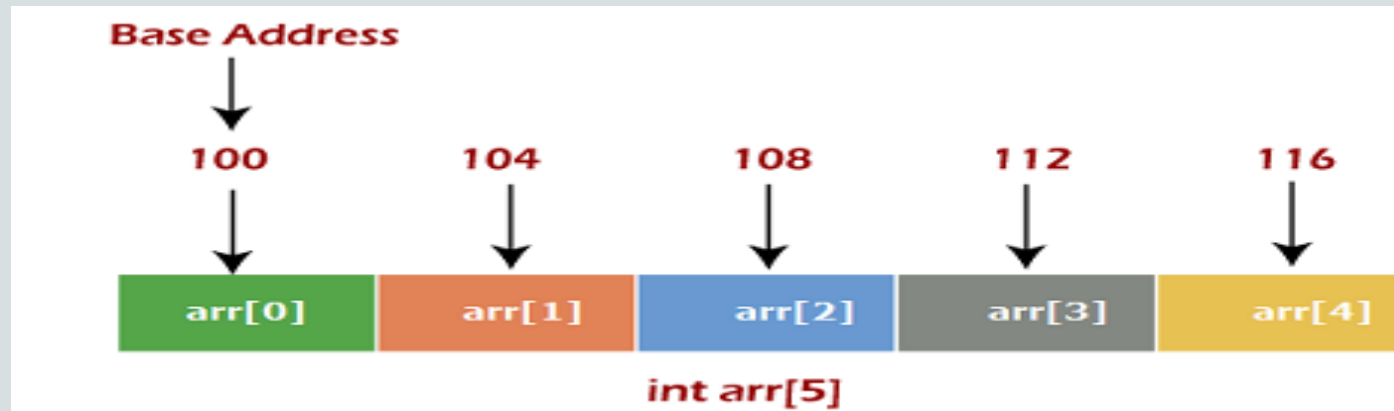
a 5 6 10 13 56 76 1 2 4 8 ✓

b 'a' 'b' 'c' 'd' 'e' ✓

c 'a' 'b' 1 5.6 'e' 34 2 3 ✗

Memory Allocation & Indexing

- **Memory allocation of an array**
- All the data elements of an array are stored at contiguous locations in the main memory. The name of the array represents the base address or the address of the first element in the main memory. Each element of the array is represented by proper indexing.
- **Indexing in the array**
- We can identify and access each value with the help of an index number in an array.
- For example, `arr[1]` will be used to access the second element of the array in zero-based indexing.
- The array indexing can be of the following types:
- **0-based indexing**
- If the first value of the array gets the index 0, it is called 0-based indexing. Most of the programming languages follow 0-based indexing.
- **1-based indexing**
- If the array index starts from 1, it is called 1-based indexing. It means the first value of the array will be assigned index 1.
- **N-based indexing**
- When the index of the first element can be anything, even negative values, it is called N-based indexing.



Name

Elements

int array [10] = { 35, 33, 42, 10, 14, 19, 27, 44, 26, 31 }

Type Size

The diagram shows the declaration and initialization of an array. The declaration 'int array [10]' is shown with 'int' as the Type and '10' as the Size. The initialization '{ 35, 33, 42, 10, 14, 19, 27, 44, 26, 31 }' is shown with a blue bracket above it labeled 'Elements'. The entire declaration and initialization are shown in red text.

Operations on 1-D & 2-D Arrays

- One-Dimensional Array

- A linearly organized collection of identical data type elements is referred to as a One-Dimensional (1D) array. A 1D array's items are kept in a single and continuous region in memory, and each element is accessible via its index. An array's initial element always has an index 0, and its last element always has an index $n-1$, where n is the array's size.

- Two-Dimensional Array

- A 2D array is a group of identical data-type components arranged in a grid-like pattern. Essentially, it is an array of arrays, where each array corresponds to a grid row. A 2D array's items are kept in a single and continuous area in memory, and each one may be reached using its row and column indices. The row number is represented by the first index, and the column number is represented by the second index.

- Basic Operations in Arrays

- The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update. These operations are usually performed to either modify the data in the array or to report the status of the array.
- Following are the basic operations supported by an array.

- **Traverse** - print all the array elements one by one.
- **Insertion** - Adds an element at the given index.
- **Deletion** - Deletes an element at the given index.
- **Search** - Searches an element using the given index or by the value.
- **Update** - Updates an element at the given index.
- **Display** - Displays the contents of the array.

- Insertion Operation

- In the insertion operation, we are adding one or more elements to the array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. This is done using input statements of the programming languages.

- Algorithm

- Following is an algorithm to insert elements into a Linear Array until we reach the end of the array -

1. Start
2. Create an Array of a desired datatype and size.
3. Initialize a variable 'i' as 0.
4. Enter the element at ith index of the array.
5. Increment i by 1.
6. Repeat Steps 4 & 5 until the end of the array.
7. Stop

- Deletion Operation

- In this array operation, we delete an element from the particular index of an array. This deletion operation takes place as we assign the value in the consequent index to the current index.

- Algorithm

- Consider LA is a linear array with N elements and K is a positive integer such that $K \leq N$. Following is the algorithm to delete an element available at the Kth position of LA.

- 1. Start
- 2. Set $J = K$
- 3. Repeat steps 4 and 5 while $J < N$
- 4. Set $LA[J] = LA[J + 1]$
- 5. Set $J = J + 1$
- 6. Set $N = N - 1$
- 7. Stop

- Search Operation

- Searching an element in the array using a key; The key element sequentially compares every value in the array to check if the key is present in the array or not.

- Algorithm

- Consider LA is a linear array with N elements and K is a positive integer such that $K \leq N$. Following is the algorithm to find an element with a value of ITEM using sequential search.
- 1. Start
- 2. Set $J = 0$
- 3. Repeat steps 4 and 5 while $J < N$
- 4. IF $LA[J]$ is equal to ITEM, THEN GOTO STEP 6
- 5. Set $J = J + 1$
- 6. PRINT J, ITEM
- 7. Stop

- Traversal Operation
- This operation traverses through all the elements of an array. We use loop statements to carry this out.
- Algorithm
- Following is the algorithm to traverse through all the elements present in a Linear Array

1. Start
2. Initialize an Array of certain size and datatype.
3. Initialize another variable 'i' with 0.
4. Print the ith value in the array and increment i.
5. Repeat Step 4 until the end of the array is reached.
6. End

- Update Operation

- Update operation refers to updating an existing element from the array at a given index.

- Algorithm

- Consider LA is a linear array with N elements and K is a positive integer such that $K \leq N$. Following is the algorithm to update an element available at the Kth position of LA.

- 1. Start
- 2. Set $LA[K-1] = \text{ITEM}$
- 3. Stop

- Display Operation

- This operation displays all the elements in the entire array using a print statement.

- Algorithm

- Consider LA is a linear array with N elements. Following is the algorithm to display an array elements.

- 1. Start
- 2. Print all the elements in the Array
- 3. Stop

- 2D Array
- 2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.
- However, 2D arrays are created to implement a relational database look alike data structure. It provides ease of holding bulk of data at once which can be passed to any number of functions wherever required.
- How to declare 2D Array
- The syntax of declaring two-dimensional array is very much like that of a one-dimensional array, given as follows.

```
int arr[max_rows][max_columns];
```

	0	1	2		n-1
0	a[0][0]	a[0][1]	a[0][2]		a[0][n-1]
1	a[1][0]	a[1][1]	a[1][2]		a[1][n-1]
2	a[2][0]	a[2][1]	a[2][2]		a[2][n-1]
3	a[3][0]	a[3][1]	a[3][2]		a[3][n-1]
4	a[4][0]	a[4][1]	a[4][2]		a[4][n-1]
.	.	.	.		.
.	.	.	.		.
n-1	a[n-1][0]	a[n-1][1]	a[n-1][2]		a[n-1][n-1]

a[n][n]

- Common Operations on Two-dimensional Arrays

- 1. Accessing Elements

- Accessing elements in a two-dimensional array involves specifying the row and column indices. For instance, to access the element at the second row and third column in the above matrix, you would use `matrix[1][2]`, considering that array indices typically start from 0.

- 2. Insertion and Deletion

- Initializing a two-dimensional array involves assigning values to its elements. For instance:
- Elements can also be modified or inserted using array indices.

- 3. Updating Elements

- Modifying the value of an existing element is a straightforward operation. Simply access the element using its indices and assign a new value.

- 4. Traversing

- Transposing a matrix involves swapping its rows with columns. For an array A , the transpose A^T is obtained by swapping $A[i][j]$ with $A[j][i]$ for all valid indices i and j .

- 5. Row and Column Operations:

- Operations on entire rows or columns are common. For example, summing the values in a row or column, finding the minimum or maximum element, or swapping two rows/columns.

- 6. Transpose:

- Transposing a matrix involves swapping its rows with columns. For an array A , the transpose A^T is obtained by swapping $A[i][j]$ with $A[j][i]$ for all valid indices i and j .

- 7. Matrix Multiplication:

- In linear algebra, multiplying two matrices is a common operation. For two matrices A and B , the product $C = A * B$ is computed as $C[i][j] = \sum(A[i][k] * B[k][j])$ for all valid indices i, j , and k .

- 8. Rotation:

- Rotating a matrix involves rearranging its elements to achieve a rotation effect. Common rotations include 90 degrees clockwise/counterclockwise and 180 degrees.

Applications of Arrays

- 1. Matrices and Graphs:

- Two-dimensional arrays are commonly used to represent matrices in mathematical operations, such as matrix multiplication.
- Adjacency matrices in graph theory use two-dimensional arrays to represent connections between vertices in a graph.

- 2. Dynamic Programming:

- Dynamic programming algorithms often utilize two-dimensional arrays to store intermediate results efficiently, reducing redundant computations.

- 3. Image Processing:

- In image processing, two-dimensional arrays are employed to represent pixels in an image, facilitating various manipulations and transformations.

- 4. Board Games:

- Games like chess or tic-tac-toe can be modeled using two-dimensional arrays to represent the game board.

Sparse Matrices

- Matrix types having most of their elements set to zero are called sparse matrices. In other terms, the definition of a sparse matrix is one in which the number of 0 entries is greater than the number of non-zero elements.
- A sparse matrix is defined as a 2-D matrix representing the total values of $m \times n$ and is composed of n columns and m rows. A matrix is referred to as sparse if the majority of its elements have a value of 0.
- The use of a sparse matrix over a basic matrix is executed as
- Storage: Since fewer non-zero elements exist than zeroes, fewer elements can be stored in a given amount of memory.
- Computing time: By logically creating a data structure that only traverses non-zero components, computation time can be minimized. The basic representation of a sparse matrix is given below:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Row (m) →

↓ Columns (n)

- Representation of Sparse Matrix

- The triplets—columns, rows, and values—can be used to store the non-zero elements of a sparse matrix. The following list includes two ways to express the sparse matrix:

- Array representation
- Linked list representation

- Array Representation of Sparse Matrix

- It wastes a lot of memory to represent a sparse matrix using a 2D array. This is due to the fact that 0s in a matrix are useless, making it wasteful to store them with non-zero elements. We can only keep non-zero elements to prevent this kind of waste. The storage space and traversal time decrease if we save non-zero elements.
- There are basically 3 fields utilized in the 2D array form of the sparse matrix, and they are as follows:
- Row: The index of the row in the matrix where a non-zero component is present.
- Column: It is defined as the index of the matrix column containing the non-0 element.
- Value: It refers to the non-0 value element at the index.

ROW	COL	VALUE
-----	-----	-------

Let's use the example below to illustrate how an array represents a sparse matrix:

Sparse Matrix →

	0	1	2	3
0	0	4	0	5
1	0	0	3	6
2	0	0	2	0
3	2	0	0	0
4	1	0	0	0

- A 5×4 sparse matrix with 7 non-zero along with 13 – zero elements can be seen in the image above. The above matrix requires $5 \times 4 = 20$ bytes of memory. The wastage of space will rise as the matrix size increases.

The representation of the matrix in tabular form will be:

Table Structure		
Row	Column	Value
0	1	4
0	3	5
1	2	3
1	3	6
2	2	2
3	0	2
4	0	1
5	4	7

The first column in the above picture denotes the rows, and the 2nd and the 3rd columns will be the non-zero values. These triplets are shown in the table's first row. The initial triplet indicates that value four is kept in the first column and the first row. Similar to the first triplet, the second shows that value 5 is kept in the first row and third column. The triplets will represent the non-zero matrix elements' stored locations similarly.

The total non-zero items in the specified sparse matrix define the table size. The memory space required by the above table, $8 \times 3 = 24$, is greater than that required by a sparse matrix.

- **Linked List Representation of Sparse Matrix**

- The sparse matrix represented by a linked list provides the benefit of using a linked list instead of an array for representing a sparse matrix is that it's simpler to add or remove nodes from a linked list than from an array.
- A node present in the representation of a sparse matrix via a linked list has four fields as opposed to three in the array representation. The linked list's four fields are listed as follows:
- Row: It depicts the representation of the position of the non-zero elements in the row at the specified index.
- **Column**: It serves as a placeholder for the non-zero element's column index.
- **Value**: This term refers to the non-zero element at the index (row as well as column).
- **Next node**: It is defined as the address of the following node once it is stored.

Node Structure

Row	Column	Value	Pointer to Next Node
-----	--------	-------	----------------------

The node is represented by each row in the output. In each row, the first element denotes the non-zero element's row index location, the second element denotes its column index location, and the 3rd element denotes the actual non-zero element.

String Manipulation Using Array

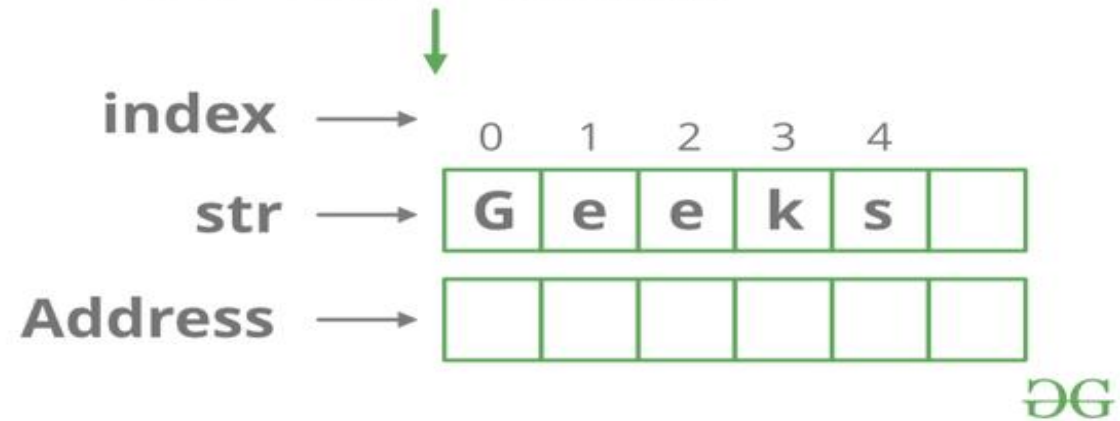
Strings are considered a data type in general and are typically represented as arrays of bytes (or words) that store a sequence of characters. Strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a special character '\0'.

- **How is String represented in Memory?**

A string can be referred to either using a character pointer or as a character array. When strings are declared as character arrays, they are stored like other types of arrays. For example, if `str[]` is an auto variable then the string is stored in the stack segment, if it's a global or static variable then stored in the data segment, etc.

String

`char str[] = "Geeks"`



- **String Operations:**
- Strings support a wide range of operations, including concatenation, substring extraction, length calculation, and more. These operations allow developers to manipulate and process string data efficiently.
- Below are fundamental operations commonly performed on strings in programming.
- **Concatenation:** Combining two strings to create a new string.
- **Length:** Determining the number of characters in a string.
- **Access:** Accessing individual characters in a string by index.
- **Substring:** Extracting a portion of a string.
- **Comparison:** Comparing two strings to check for equality or order.
- **Search:** Finding the position of a specific substring within a string.
- **Modification:** Changing or replacing characters within a string.